

Go-reloaded

Bon retour parmi nous. Félicitations pour ton admission. Je savais que tu y arriverais. Il est maintenant temps de passer aux projets.

Objectifs

Dans ce projet, tu vas réutiliser certaines des fonctions que tu as écrites dans ton ancien dépôt. Tu t'en serviras pour créer un outil simple de complétion, d'édition et d'autocorrection de texte.

Je te conseille de créer tes propres tests, pour toi-même.

Introduction

- Ton projet doit être écrit en **Go**.
- Le code doit respecter les [bonnes pratiques](#).
- Il est recommandé d'avoir des **fichiers de test** pour les [tests unitaires](#).

L'outil que tu vas construire recevra en arguments le nom d'un fichier contenant un texte à modifier (l'entrée) et le nom du fichier dans lequel le texte modifié devra être écrit (la sortie). Voici la liste des modifications que ton programme doit effectuer :

- Chaque occurrence de `(hex)` doit remplacer le mot qui la précède par sa version décimale (dans ce cas, le mot sera toujours un nombre hexadécimal). (Ex : "1E (hex) files were added" -> "30 files were added")
- Chaque occurrence de `(bin)` doit remplacer le mot qui la précède par sa version décimale (dans ce cas, le mot sera toujours un nombre binaire). (Ex : "It has been 10 (bin) years" -> "It has been 2 years")
- Chaque occurrence de `(up)` convertit le mot qui la précède en majuscules. (Ex : "Ready, set, go (up) !" -> "Ready, set, GO!")
- Chaque occurrence de `(low)` convertit le mot qui la précède en minuscules. (Ex : "I should stop SHOUTING (low)" -> "I should stop shouting")
- Chaque occurrence de `(cap)` convertit le mot qui la précède avec une majuscule initiale. (Ex : "Welcome to the Brooklyn bridge (cap)" -> "Welcome to the Brooklyn Bridge")
 - Pour `(low)` , `(up)` et `(cap)` , si un nombre apparaît à côté, comme ceci : `(low, <number>)` , le nombre de mots précédents indiqué est converti en minuscules, en majuscules ou avec une majuscule initiale selon le cas. (Ex : "This is so exciting (up, 2)" -> "This is SO EXCITING")

- Chaque signe de ponctuation . , , ! , ? , : et ; doit être collé au mot précédent et séparé du mot suivant par un espace. (Ex : "I was sitting over there ,and then BAMM !!" -> "I was sitting over there, and then BAMM!!").
 - Sauf s'il s'agit de groupes de ponctuation comme ... ou !? . Dans ce cas, le programme doit formater le texte comme dans l'exemple suivant : "I was thinking ... You were right" -> "I was thinking... You were right".
- Le signe de ponctuation ' sera toujours accompagné d'une autre occurrence de lui-même, et les deux doivent être placés à droite et à gauche du mot situé entre eux, sans aucun espace. (Ex : "I am exactly how they describe me: ' awesome '" -> "I am exactly how they describe me: 'awesome'")
 - S'il y a plus d'un mot entre les deux signes ' ' , le programme doit placer les signes contre les mots correspondants. (Ex : "As Elton John said: ' I am the most well-known homosexual in the world '" -> "As Elton John said: 'I am the most well-known homosexual in the world'")
- Chaque occurrence de a doit être transformée en an si le mot suivant commence par une voyelle (a , e , i , o , u) ou par un h . (Ex : "There it was. A amazing rock!" -> "There it was. An amazing rock!").

Packages autorisés

- Les packages de la [bibliothèque standard de Go](#) sont autorisés.

Utilisation

```
$ cat sample.txt
it (cap) was the best of times, it was the worst of times (up) , it was the age of wisdom, it
$ go run . sample.txt result.txt
$ cat result.txt
It was the best of times, it was the worst of TIMES, it was the age of wisdom, It Was The Age
$ cat sample.txt
Simply add 42 (hex) and 10 (bin) and you will see the result is 68.
$ go run . sample.txt result.txt
$ cat result.txt
Simply add 66 and 2 and you will see the result is 68.
$ cat sample.txt
There is no greater agony than bearing a untold story inside you.
$ go run . sample.txt result.txt
$ cat result.txt
There is no greater agony than bearing an untold story inside you.
$ cat sample.txt
Punctuation tests are ... kinda boring ,what do you think ?
$ go run . sample.txt result.txt
$ cat result.txt
Punctuation tests are... kinda boring, what do you think?
```

Ce projet t'aidera à découvrir :

- L'API du système de fichiers de Go (**fs**)
- La manipulation de chaînes de caractères et de nombres